

Two things that changed today.

-
- 01 Same code-review workload, same platform: the price list spans 267× from cheapest to most expensive.
-
- 02 Ollama now honours the sampling defaults declared inside the model file. Local outputs may shift.

Same code-review workload, same platform: the price list spans 267x from cheapest to most expensive.

- budget the prompt: shave tokens, trim the system message
- + budget the model: the same month costs \$0.30 or \$80.00

WHY IT MATTERS

Above a certain volume the model you pick decides the bill, not how tightly you write the prompt.

WHO SHOULD CARE

Anyone running a high-volume automated workload: reviews, summaries, extraction, agent loops.

REPRODUCE IT

AWAITING CI

Today's real price list, one workload, sorted by monthly cost.

```
import json, pathlib
M = json.loads((pathlib.Path(__file__).parent / "prices.json").read_text())["models"]

IN_TOK, OUT_TOK, PER_MONTH = 30_000, 2_000, 200      # one code review, and how many a month

def monthly(m):
    return (IN_TOK * m["prompt"] + OUT_TOK * m["completion"]) * PER_MONTH
...
lo, hi = monthly(rows[0]), monthly(rows[-1])
print(f"\nsame workload, same platform: ${lo:.2f} vs ${hi:.2f} per month  ->  {hi/lo:.0f}x")
```

inception/mercury-2.5-preview	260k ctx	\$	0.30 / month
z-ai/glm-5.3-flash	1310k ctx	\$	0.55 / month
ibm-granite/granite-4.2-8b	131k ctx	\$	0.66 / month
meta/muse-spark-1.2-contributor	1048k ctx	\$	0.68 / month
qwen/qwen3.8-flash	1000k ctx	\$	1.09 / month
deepseek/deepseek-v4-flash-vision-exp	1048k ctx	\$	1.58 / month
tencent/hy4-preview	1048k ctx	\$	6.00 / month
anthropic/claude-fable-5.1	1000k ctx	\$	80.00 / month

same workload, same platform: \$0.30 vs \$80.00 per month -> 267x

\$0.30 → \$80.00

the same 200-review month, cheapest against most expensive
on today's list — 267×



Ollama now honours the sampling defaults declared inside the model file. Local outputs may shift.

- the runner's own defaults, whatever the file declared
- + the defaults the model author shipped inside the GGUF

WHY IT MATTERS

If you relied on the runner's own defaults, this upgrade changes your output distribution.

WHO SHOULD CARE

Anyone running local models through Ollama in something that has to be reproducible.

Same logits, two default sets, two different models in practice.

```
logits = np.array([4.1, 3.9, 3.4, 2.8, 2.7, 2.1, 1.6, 1.2, 0.9, 0.4, -0.6, -1.8])

def distribution(temperature, top_p):
    p = np.exp((logits - logits.max()) / temperature); p /= p.sum()
    order = np.argsort(-p)
    keep = order[:np.searchsorted(np.cumsum(p[order]), top_p) + 1]
    q = np.zeros_like(p); q[keep] = p[keep]
    return q / q.sum()
```

```
generic fallback      temp=1.0 top_p=1.00    top token 0.314    tokens reachable 12/12
declared in the file  temp=0.6 top_p=0.90    top token 0.467    tokens reachable 4/12
```

```
total variation distance between the two: 0.230
same logits, same model weights - only the defaults differ
```

12 → 4

tokens the model can pick at one step — same weights, only the defaults differ

TAKEAWAY

The model you pick decides the bill more than the prompt you write.
And if you run models locally, check your defaults after upgrading.

[FULL CODE IN DESCRIPTION](#)

[A NEW DIFF EVERY WEEKDAY](#)